Z3-Noodler 1.5.4
System description for SMT-COMP 2026

David Chocholatý[1], Vojtěch Havlena[1], Michal Hečko[1],
Lukáš Holík[1,2], Ondřej Lengál[1], Juraj Síč[1]

[1] Brno University of Technology, Brno
[2] The Technical Faculty of IT and Design, Aalborg University, Aalborg, Denmark

Abstract

This is a brief overview of the string solver Z3-Noodler[1] 1.5.4 entering SMT-COMP 2026. It is based on the SMT solver Z3 [1] in which it replaces the string theory solver. Z3-Noodler combines multiple decision procedures for efficient handling of different fragments of string constraints with the stabilization-based procedure as the main one [2, 3]. Z3-Noodler heavily relies on nondeterministic finite automata (NFAs) whose handling is provided by the automata library Mata [4].

1 Overview

Z3-Noodler[1] is a string solver that targets string constraints such as those that occur in the analysis of programs, regular filters, policy descriptions, etc. It is built on top of the SMT solver Z3 [1] and the automata library Mata[2]. The core of the string solver is the *stabilization-based* decision procedure from [2, 3] aiming at solving string (dis)equations with length and regular constraints.

Z3-Noodler replaces the string theory solver of Z3 with a new theory solver that uses the infrastructure of Z3, in particular string theory rewriter for lightweight simplification of input formula and linear arithmetic solver (LIA) for solving length constraints. Z3-Noodler's string theory solver then interacts with the core of Z3 in a usual manner: based on an input conjunction of string constraints the solver reduces the conjunction to a LIA formula over string lengths and returns it to Z3 as a theory lemma. To speed up obtaining a satisfiable case, the solver generates parts of the LIA formula lazily, when necessary. Z3-Noodler implements multiple decision procedures, which use the Mata library for handling NFAs. Z3-Noodler is implemented in C++, as well as Z3 and Mata. For a more detailed description of Z3-Noodler's design, we refer to [5, 6], papers on which this description is based.

2 Core of the String Solver

From a high-level point of view the string solver of Z3-Noodler consists of several parts: (i) interface interacting with Z3's core, (ii) axiom saturation for string functions/predicates, (iii) preprocessing of string constraints, and (iv) decision procedures for satisfiability checking of a conjunction of string atoms. Below, we give a brief description of some important features of the string solver.

Axiom Saturation. In order to maximize the benefits of Z3's internal LIA solver during generation of a satisfiable assignment, we saturate the input formula with length-aware axioms and specific axioms for string predicates/functions. For instance $|t_1.t_2| = |t_1| + |t_2|$ where t_1, t_2 are string terms. We also use different saturation rules for functions/predicates instantiated by concrete values, e.g., the axiom $substr(t_1, 4, 1) = at(t_1, 4)$.

Preprocessing. The core string solver uses a set of simple rewriting rules that eliminate trivial equations such as $x = y$, simplify equations when a string variable is known to equal the empty string, transform equations into regular membership constraints when possible ($x = uv$ becomes $x \in L_u \cdot L_v$ if u, v do not appear elsewhere and are constrained by the languages L_u and L_v, respectively), simplify equations such as $xyz = xuz$ into $y = u$, and many others. Besides the semantics preserving rewriting rules, we use one under-approximating rule that replaces a membership of a variable in a co-finite language by a length constraint that excludes all the lengths of words outside the language. Each decision procedure uses a tailored sequence of preprocessing rules that are beneficial for the particular decision procedure.

Combination of decision procedures. From a high-level point of view, Z3-Noodler implements multiple complementary decision procedures. A particular decision procedure is selected according to features of the input string constraints [6]. The main decision procedure is the *stabilization-based* procedure [2, 3] for efficient solving of string (dis)equations with length and regular constraints. The procedure iteratively

[1] `https://github.com/VeriFIT/z3-noodler`
[2] `https://github.com/VeriFIT/mata`

refines the language of each variable (represented by an NFA) according to word equations until the stabilization condition is met in a way that the concatenation of languages corresponding to the left- and right-hand sides of each equation is equal. When the stability is achieved, length constraints constructed from variable languages are passed to the LIA solver. The language refinement is done through *noodlification*, which is an automata operation for generating feasible variable alignments. The algorithm is complete for the *chain-free* [7] combinations of equations, length and regular constraints, together with unrestricted disequations, making it the largest known decidable fragment of these types of constraints.

For efficient handling of *quadratic* equations (at most two occurrences of each variable) with length constraints, Z3-Noodler implements Nielsen transformation [8]. The algorithm constructs a graph corresponding to the system and reasons about it to determine if the input formula is satisfiable or not [9, 10]. If the system contains length variables, we also create a counter automaton corresponding to the Nielsen graph (in a similar way as in [11]). In the subsequent step, we contract edges, saturating the set of self-loops and, finally, we iteratively generate flat counter sub-automata (a flat counter automaton only allows cycles that are self-loops), which are later transformed into LIA formulae describing lengths of all possible solutions. We also employ optimizations, such as state-space pruning, prioritization of promising paths, etc. In Z3-Noodler we also use this procedure for general cyclic equations (without guarantee of termination).

Although the stabilization-based procedure can be fast, it might suffer from an explosion in the number of noodles, especially for large systems of equations with unrestricted variables. In order to efficiently deal with these equations, Z3-Noodler uses *length-based decision procedure* allowing us to symbolically encode alignments using LIA formulae [6].

For efficient handling of difficult regular constraints, Z3-Noodler implements construction of NFAs from regular expressions with eager minimization/simulation-based reduction. To avoid useless constructions, we sort regular expressions according to expected size of the corresponding NFA and eagerly combine obtained NFAs to get the answer as soon as possible [6].

3 Extended String Constraints

Z3-Noodler offers support for various extended string constraints.

String-int conversions. In order to support string-integer conversions (`str.from_int`, `str.to_int`, `str.from_code`, and `str.to_code`), Z3-Noodler builds upon the stabilization-based procedure by reasoning over stabilized regular languages of converted variables. After a stable solution is found, it builds a compact LIA encoding of possible numeric values and lengths using interval abstractions, while also relating values of multiple conversions over shared variables [12]. The method is precise when stabilized languages are finite, and for infinite languages it uses a length-bounded underapproximation.

Negated containment. Difficult positional constraints, such as `str.¬contains`, are handled via tag automata in Z3-Noodler. After a stable solution is found, the semantics of `str.¬contains` are represented by a tag automaton enriched with additional quantified LIA formulae describing all models of the predicates provided that the stable languages are flat [13].

Transducer constraints. Z3-Noodler extends stabilization to transducer constraints needed for string functions such as `str.replace_all` and `str.replace_re_all`, while combining them with equations and length/regular constraints in the chain-free fragment [14]. To avoid the blow-up of alignment splitting, it applies stabilization whenever possible and eliminates concatenation by transducer noodlification only in subproblems where interaction with length constraints is necessary. The solver also uses practical heuristics, including simplification of homomorphic transducers, rewriting constant input/output transducers to equations, and attempts to conclude (un)satisfiability for cyclic transducer constraints.

4 Z3-Noodler at SMT-COMP 2026

We are submitting version 1.5.4 of Z3-Noodler to participate in the single-query and incremental track divisions QF_S, QF_SLIA, and QF_SNIA. The submitted version is based on Z3 version 4.15.8 and Mata version 1.32.0.

References

[1] Leonardo Mendonça de Moura and Nikolaj Bjørner. "Z3: An Efficient SMT Solver". In: *TACAS'08*. Vol. 4963. LNCS. Springer, 2008, pp. 337–340. URL: https://doi.org/10.1007/978-3-540-78800-3%5C_24.

[2] František Blahoudek et al. “Word Equations in Synergy with Regular Constraints”. In: *Formal Methods*. Ed. by Marsha Chechik, Joost-Pieter Katoen, and Martin Leucker. Cham: Springer International Publishing, 2023, pp. 403–423. ISBN: 978-3-031-27481-7.

[3] Yu-Fang Chen et al. “Solving String Constraints with Lengths by Stabilization”. In: *Proc. ACM Program. Lang.* 7.OOPSLA2 (Oct. 2023). DOI: `10.1145/3622872`.

[4] David Chocholatý et al. “Mata: A Fast and Simple Finite Automata Library”. In: *Tools and Algorithms for the Construction and Analysis of Systems*. Cham: Springer Nature Switzerland, 2024, pp. 130–151.

[5] Yu-Fang Chen et al. “Z3-Noodler: An Automata-based String Solver”. In: *Tools and Algorithms for the Construction and Analysis of Systems*. Cham: Springer Nature Switzerland, 2024, pp. 24–33. ISBN: 978-3-031-57246-3.

[6] David Chocholatý et al. “Z3-Noodler 1.3: Shepherding Decision Procedures for Strings with Model Generation”. In: *Tools and Algorithms for the Construction and Analysis of Systems*. Ed. by Arie Gurfinkel and Marijn Heule. Cham: Springer Nature Switzerland, 2025, pp. 23–44. ISBN: 978-3-031-90653-4.

[7] Parosh Aziz Abdulla et al. “Chain-Free String Constraints”. In: *Automated Technology for Verification and Analysis - 17th International Symposium, ATVA 2019, Taipei, Taiwan, October 28-31, 2019, Proceedings*. Ed. by Yu-Fang Chen, Chih-Hong Cheng, and Javier Esparza. Vol. 11781. Lecture Notes in Computer Science. Springer, 2019, pp. 277–293. DOI: `10.1007/978-3-030-31784-3\_16`. URL: `https://doi.org/10.1007/978-3-030-31784-3%5C_16`.

[8] Jakob Nielsen. “Die Isomorphismen der allgemeinen, unendlichen Gruppe mit zwei Erzeugenden”. In: *Mathematische Annalen* 78.1 (1917), pp. 385–397.

[9] John Michael Robson and Volker Diekert. “On quadratic word equations”. In: *Annual Symposium on Theoretical Aspects of Computer Science*. Springer. 1999, pp. 217–226.

[10] Yu-Fang Chen et al. “A symbolic algorithm for the case-split rule in solving word constraints with extensions”. In: *Journal of Systems and Software* 201 (2023), p. 111673. ISSN: 0164-1212. DOI: `https://doi.org/10.1016/j.jss.2023.111673`. URL: `https://www.sciencedirect.com/science/article/pii/S0164121223000687`.

[11] Anthony W. Lin and Rupak Majumdar. “Quadratic Word Equations with Length Constraints, Counter Systems, and Presburger Arithmetic with Divisibility”. In: *Automated Technology for Verification and Analysis*. Cham: Springer International Publishing, 2018, pp. 352–369. ISBN: 978-3-030-01090-4.

[12] Vojtěch Havlena et al. “Cooking String-Integer Conversions with Noodles”. In: *27th International Conference on Theory and Applications of Satisfiability Testing (SAT 2024)*. Ed. by Supratik Chakraborty and Jie-Hong Roland Jiang. Vol. 305. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, 14:1–14:19. ISBN: 978-3-95977-334-8. DOI: `10.4230/LIPIcs.SAT.2024.14`. URL: `https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.SAT.2024.14`.

[13] Yu-Fang Chen et al. “A Uniform Framework for Handling Position Constraints in String Solving”. In: *Proc. of 46th ACM SIGPLAN Conference on Programming Language Design and Implementation - PLDI’25*. PACMPL 9 (169). ACM, 2025.

[14] David Chocholatý et al. “String Solving with Stabilization and Transducers”. In: *To appear in Proc. of CAV’26*. Lecture Notes in Computer Science. Springer, 2026.